LINUX SERVER ATTACK RESIDUE & FORENSIC RESPONSE SYSTEM

PROJECT SYNOPSIS

A report submitted in partial fulfillment of the requirements

for the degree of Bachelor of Technology in

Computer Science & Engineering / Cyber Security

Submitted by

Aman Sharma — 2400320100161

Under the guidance of

Sarath G

Cybersecurity Trainer & Consultant

Department of Computer Science & Engineering

ABES Engineering College, Ghaziabad

Dr. A.P.J. Abdul Kalam Technical University, Lucknow — Session 2026-27

1. Introduction / Background

Modern Linux cloud servers face frequent targeted attacks where adversaries gain initial access via web vulnerabilities, plant persistence mechanisms (such as unauthorized systemd services, web shells, and cron jobs), and clean up transient tools to evade traditional real-time security monitoring. Attack Residue Detection focuses on identifying the lingering physical artifacts and state deviations left behind post-exploitation.

2. Problem Statement

Traditional log-based Intrusion Detection Systems (IDS) generate heavy false positive alert floods during routine system administration and often miss stealthy post-exploitation modifications once network traffic ceases. There is a need for a lightweight, zero-dependency residue detection engine that compares host state against a trusted baseline and correlates artifacts with kernel audit logs.

3. Objectives

1. Baseline Recording: Cryptographically hash and record baseline host state (files, unit files, listening ports, user accounts).
2. Controlled Simulation: Execute realistic multi-stage web compromise scenarios safely on target VMs.
3. State Diffing & Residue Detection: Identify unlisted executables, modified web pages, unauthorized services, and new ports.
4. Forensic Correlation: Map detected residues to raw Auditd system call logs and Nginx web access logs.
5. Automated Containment & Live UI: Neutralize unauthorized processes/services and visualize findings on a live dashboard.

4. Scope of the Project

The system targets Linux distributions (Ubuntu 26.04 LTS) running web services (Nginx, PHP-FPM). It covers file integrity monitoring, systemd unit file verification, listening port auditing, process inspection, Auditd syscall event parsing, and automated response.

5. Proposed System / Methodology

The proposed methodology follows a 4-phase lifecycle:

• Phase 1 (Baseline): Capture trusted JSON state manifest with SHA256 hashes.

• Phase 2 (Compromise & Scan): Simulate attack and perform state diffing to generate Residue objects.

• Phase 3 (Forensic Correlation): Correlate Residue objects with Auditd logs (`comm`, `exe`, `path`, `pid`) and Nginx access logs.

• Phase 4 (Containment & UI): Apply automated remediation (kill processes, disable services, restore files) and serve web UI.

6. Modules / Module Description

• Module 1 (Baseline Engine): Creates and cryptographically verifies trusted state manifests.

• Module 2 (Residue Detector): Diffs host state and calculates risk severity scores (1-10).

• Module 3 (Forensic Correlator): Parses `/var/log/audit/audit.log` and Nginx logs to reconstruct attack paths.

• Module 4 (Containment Engine): Neutralizes malicious artifacts and verifies host recovery.

• Module 5 (Presentation Dashboard): Serves dark-mode web monitoring UI.

7. Hardware & Software Requirements

• Target Host: Oracle Cloud Ubuntu 26.04 LTS (`bot-farm`), 1 vCPU, 1GB RAM.

• Attacker Workstation: Kali Linux Workstation.

• Software Stack: Python 3.14 (Stdlib), Auditd 3.x, Nginx 1.26, Systemd, Bash.

8. Feasibility Study

• Technical: Uses native Python standard library and built-in Linux kernel audit facilities (zero external dependencies).

• Economic: 100% open-source software stack with zero licensing cost.

• Operational: Seamless integration into standard Linux system administration workflows.

9. Expected Outcomes

• Sub-second residue detection latency (<1.0s).
• 100% detection rate on planted web shell and systemd service persistence.
• Zero false positives on authorized maintenance operations.
• Real-time web presentation dashboard with live evidence timeline.

10. Timeline / Project Plan

• Day 1: System setup, baseline engine creation, and compromise simulation.
• Day 2: Residue detection logic and forensic audit log correlation engine.
• Day 3: Automated containment engine, post-response verification, and web UI.
• Day 4: Documentation (SRS, Synopsis, UseCases, PPT) and live demo dry run.

11. References

[1] IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
[2] Linux Audit Subsystem Documentation (man 8 auditd).
[3] MITRE ATT&CK Framework: Persistence (T1543) and Web Shells (T1505.003).